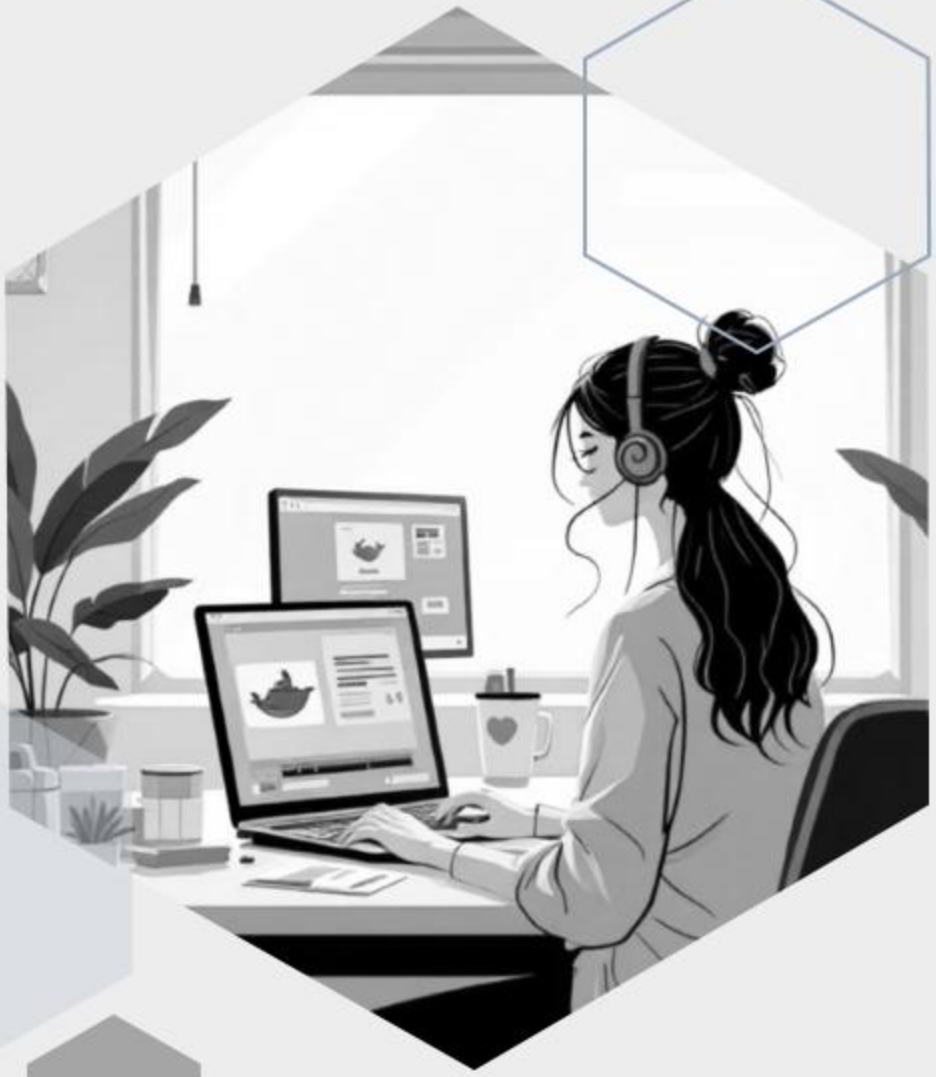
Docker & Conteneurisation

Departement : Génie Informatique

Spécialité : Développement Web



Y.NDIONE (Formateur certifié)

Mail: youcomputing@gmail.com

Chapitre 2

Construire son propre image Docker

O P : A la fin de ce chapitre , l'étudiant sera capable de :

- ✓ Comprendre le rôle d'un Dockerfile
- ✓ Construire une image personnalisée Docker
- ✓ Comprendre et expliquer la notion de Layer
- ✓ Utiliser le Tagging
- ✓ Optimiser un Dockerfile

Rappel

Image → Modèle immuable

Conteneur → Instance en exécution

Provenance d'une image → Dockerfile

1

Qu'est ce qu'un Dockerfile ?

Un Dockerfile est un fichier texte contenant une suite d'instructions permettant de construire automatiquement une image Docker.

2

Structure Générale d'un Dockerfile

Instruction principale

- FROM
- WORDIR
- COPY
- RUN
- EXPOSE
- CMD
- ENV

NB : Chaque instruction crée une couche (Layer)

3 Notion de couche (Layer)

Une couche Docker (Docker layer) est une unité atomique, immuable et intermédiaire qui constitue le système de fichiers d'une image conteneurisée. Formellement, il s'agit d'un ensemble de différences (delta) par rapport à l'état précédent du système de fichiers, encapsulé sous forme d'archive (tarball) et identifié de manière unique par une empreinte cryptographique SHA256.

Chaque instruction → Couche (Layer)

Précision importante : en cas de rebuild , Docker ne reconstruit que les couches modifiées

3 Notion de couche (Layer)

Les couches Docker (layer) présente plusieurs avantages

- ✓ Mise en cache
- ✓ Réutilisation
- ✓ Optimisation des builds

4

Commande de création d'une image

```
docker built -t nom_image : version .
```

Exemple : se positionner dans le dossier cible et entrer la commande

docker built – t myApp : v1 .

NB : n'oublier pas le point à la fin

5 Optimisation d'un Dockerfile

Bonnes pratiques : Reduction considérable du temps de build

- ✓ Utiliser des images légères (Alpine)
- ✓ Regrouper les RUN
- ✓ Exploiter le cache
- ✓ Copier (COPY) et charger (RUN) les dépendances avant de copier (COPY) le reste du code source

5 Optimisation d'un Dockerfile (suite)

Exemple : Cas d'une application node

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

5 Optimisation d'un Dockerfile (suite)

Exemple : Cas d'une application node

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

6 Publier une image sur Docker HUB

Préliminaire : Créer un compte (username & password) dans Docker Hub et créer et configurer un référentiel (*repository*)

- ❑ connexion
  - → docker login -u username
- ❑ tagging
  - → docker tag mon-image : version username/nom_repository : version
- ❑ Publication sur Docker Hub
  - → docker push username/nom_repository : version

Ateliers Pratiques : Conteneurisation d'une application Python avec Flask

1. Créer une mini application Flask
2. Créer un Dockerfile pour cette application
3. Exposer le port 5000
4. Construire l'image
5. Lancer le conteneur

Merci

Y. NDIONE

youcomputing@gmail.com